Marisa Munoz
CS 4953 High Performance Machine Learning
Erv655
April 29, 2026

Implementation Paper: Algorithms 1, 2, and 3

Comparing Energy Costs of Parallel DNN Training Techniques

Note on AI Usage: Revised with AI for coding support(Claude, Anthropic) per CS 4953 LLM use policy. All experimental results are original, collected from cluster runs on NCSA Delta.

Note on Data: Finalized source code, CSV result files, power logs, and performance plots are posted to the course GitHub at github.com/marisamunoz/Green-DNN-Training-UTSA.

1. Problem

The environmental cost of training AI models has become a real issue. Data centers that run these workloads are consuming massive amounts of electricity and water. By 2025, estimates put AI's carbon footprint at the equivalent of New York City and its water usage at the scale of global bottled water consumption annually. Most of the research focus goes toward making models more accurate or faster, but very little asks: which training method actually uses less power to get there?

This project tries to answer that question in a concrete and measurable way. I'm comparing three classes of parallel DNN training techniques and measuring not just training speed but actual energy consumption. Algorithm 1 establishes the data parallelism baseline across three frameworks. Algorithm 2 measures the energy impact of reduced-precision training techniques. Algorithm 3 investigates ZeRO memory optimization to determine whether eliminating redundant GPU memory storage can reduce energy consumption. Together, the three algorithms build toward a picture of which training configuration is most environmentally sustainable.

2. Chosen Algorithms / Methods

Algorithm 1: Data Parallelism

Data parallelism is the most widely used approach for distributed DNN training and the obvious starting point for this project. The core idea is straightforward: every GPU gets a complete copy of the model, but each one processes a different chunk of the training data at the same time. At the end of each step, the gradients from all GPUs get averaged and the weights get updated. That synchronization step is what makes it work, but it also introduces communication overhead that grows as you add more GPUs and that overhead costs energy.

Rather than implementing data parallelism just once, Professor Prasad's feedback pushed me to compare how three different frameworks handle the same underlying idea. Each one uses a different communication strategy and design philosophy:

Method 1a: PyTorch Distributed Data Parallel (DDP)

PyTorch DDP is the standard baseline. Each GPU runs its own training process, and gradients are synchronized using the NCCL all-reduce operation at the end of each backward pass. DDP overlaps gradient communication with computation as soon as a layer's gradients are ready. Implementation uses torchrun to launch multiple processes (Li et al., 2020).

Method 1b: TensorFlow MirroredStrategy

TensorFlow's MirroredStrategy mirrors the model across all available GPUs and uses all-reduce to synchronize gradients. TF handles distribution at the framework level rather than the process level, which makes it easier to set up but gives less control over the underlying communication. MirroredStrategy also scales the batch size automatically by the number of GPUs.

Method 1c: AxoNN

AxoNN (Singh & Bhatele, 2022) was developed for extreme-scale training and uses asynchronous MPI-based point-to-point messaging instead of synchronous NCCL all-reduce, which lets GPUs keep computing while communication happens in the background. AxoNN was shown to outperform Megatron-LM and DeepSpeed on multi-billion parameter transformer models. For CIFAR-10/ResNet18 at our scale, it was an open question whether the asynchronous advantage would hold.

Algorithm 2: Mixed Precision and Quantization

Algorithm 2 investigates whether reducing numerical precision during training can lower energy consumption without meaningfully hurting accuracy. All experiments were run on a single NVIDIA A40 GPU to isolate the effect of precision format.

Method 2a: FP16 Mixed Precision

FP16 drops down to 16-bit for most computation during training while keeping a master copy of weights in FP32. PyTorch's Automatic Mixed Precision handles the precision switching automatically, and a GradScaler prevents gradient underflow (Micikevicius et al., 2018).

Method 2b: BF16

BF16 keeps the same exponent range as FP32 but reduces mantissa precision, making it numerically more stable than FP16. It does not require a GradScaler and is supported natively on NVIDIA Ampere GPUs including the A40.

Method 2c: INT8 Post-Training Dynamic Quantization

INT8 maps 32-bit floating point weights to 8-bit integers, reducing model size by 4x. The model is trained normally in FP32 and quantization is applied afterward without any changes to the training process.

Method 2d: Knowledge Distillation

Knowledge distillation trains a smaller student model (ResNet18) using soft probability outputs from a larger teacher model (ResNet50). The combined loss is a weighted sum of the hard label cross-entropy and the KL divergence from the teacher's soft outputs, controlled by temperature T=4 and alpha=0.7 (Hinton et al., 2015).

Note on QAT: Quantization-Aware Training was attempted but encountered a compatibility issue with PyTorch 2.6 fake quantization on the Delta cluster and is documented in the repository error logs.

Algorithm 3: ZeRO Memory Optimization

Algorithm 3 investigates DeepSpeed ZeRO (Zero Redundancy Optimizer), a memory optimization strategy that eliminates redundant tensor storage in distributed training. In standard data parallelism, every GPU holds a complete copy of optimizer states, gradients, and model parameters. ZeRO partitions these across GPUs so each one only stores its assigned share (Rajbhandari et al., 2020).

ZeRO defines three progressive stages. Stage 1 partitions optimizer states only. Stage 2 additionally partitions gradients. Stage 3 additionally partitions model parameters, enabling training of models too large to fit on a single GPU. All experiments used 2 NVIDIA A40 GPUs on NCSA Delta, with DeepSpeed implementing the partitioning and collective communication automatically.

3. Underlying Communication Patterns

All three Algorithm 1 frameworks use gradient averaging across GPUs but implement it differently. PyTorch DDP uses NCCL ring all-reduce where gradients flow around a ring topology until all GPUs have the full sum. TensorFlow MirroredStrategy also uses NCCL all-reduce under the TF runtime. AxoNN uses asynchronous MPI point-to-point where GPUs send and receive gradients without blocking.

The all-reduce communication volume per step is approximately proportional to model parameters. For ResNet18, that is roughly 11 million parameters or about 44MB in FP32. This stays constant per step regardless of GPU count, which means communication becomes a larger fraction of total time as per-step compute gets faster.

Algorithm 2 methods do not change the communication pattern. FP16 and BF16 reduce the memory footprint of activations and gradients proportionally. INT8 is applied after training. Distillation uses the same DDP all-reduce as FP16.

Algorithm 3 replaces all-reduce with two new collective operations. Reduce-scatter synchronizes and partitions gradients in one step, so each GPU receives only its designated slice. All-gather reconstructs the full parameter tensor from all GPU slices before the forward pass. Stage 3 requires all-gather for parameters on every forward and backward pass, which significantly increases communication volume compared to Stages 1 and 2.

4. Data Structures, Datasets, and Hyperparameters

Model: ResNet18, trained from scratch with no pretrained weights, modified for 10-class output to match CIFAR-10.

Dataset: CIFAR-10, 60,000 32x32 color images across 10 categories (50,000 train, 10,000 test). Each GPU receives a non-overlapping shard via DistributedSampler (PyTorch) or automatic sharding (TF/AxoNN/DeepSpeed).

Shared hyperparameters across all experiments:

- Batch size: 128 per GPU
- Learning rate: 0.1 with cosine annealing over 10 epochs
- Optimizer: SGD with momentum 0.9, weight decay 5e-4
- Epochs: 10 per run
- Each configuration run 3 times to account for variance

Algorithm 1: GPU counts tested were 1 and 2 for all three frameworks. Precision: FP32.

Algorithm 2: All experiments on 1 GPU. Distillation uses ResNet50 teacher with temperature T=4 and alpha=0.7.

Algorithm 3: All experiments on 2 GPUs. DeepSpeed ZeRO Stages 1, 2, and 3 tested.

5. Read/Write Contention and Synchronization Overheads

In DDP, the main synchronization point is the all-reduce at the end of each backward pass, which is a hard synchronization barrier before the optimizer step. DDP mitigates this by bucketing gradients and overlapping communication with computation, but the barrier still exists.

Read/write contention is relatively low in DDP because all-reduce is symmetric and no single node is a bottleneck. In AxoNN, the asynchronous model removes the hard barrier but introduces stale gradient risk if a GPU finishes a step before receiving all updates.

Memory-wise, each GPU in DDP holds the full model, optimizer states, gradients, and activations. For ResNet18 this is approximately 500MB-1GB, well within the A40's 46GB capacity.

For Algorithm 2, FP16 and BF16 reduce the memory footprint of activations by half. INT8 is applied after training and does not affect synchronization. Distillation adds a second model to GPU memory, increasing usage to approximately 462MB.

For Algorithm 3, ZeRO reduces redundant memory storage at the cost of additional communication. Stage 1 and 2 add reduce-scatter communication after the backward pass. Stage 3 adds all-gather communication before every forward pass to reconstruct parameters. At ResNet18 scale the Stage 3 all-gather overhead is significant relative to the actual compute, as evidenced by the 3x increase in epoch time compared to Stages 1 and 2.

6. Parallel Time Complexity

For N GPUs, dataset size D, and per-GPU batch size B:

- Computation per step: O(B x model_FLOPs), constant per GPU regardless of N
- Communication per step (DDP): O(P / bandwidth) where P is approximately 44MB for ResNet18
- Steps per epoch: O(D / (N x B)), decreases linearly with GPU count
- Total time per epoch: O((D / (N x B)) x (t_compute + t_communicate))

For Algorithm 2, FP16 and BF16 reduce t_compute and memory bandwidth by 2x. INT8 reduces this further by 4x. Distillation doubles t_compute per step due to the teacher forward pass.

For Algorithm 3, ZeRO Stages 1 and 2 add reduce-scatter after the backward pass with similar volume to DDP all-reduce. Stage 3 adds all-gather before every forward pass, which roughly triples t_communicate at 2 GPUs with a small model. At large GPU counts and large models, the memory savings from ZeRO would outweigh the additional communication cost, but at 2 GPUs with ResNet18 Stage 3 is dominated by communication overhead.

7. Timing and Experiment Details

Each configuration was run 3 times to average out variance. Parameters varied:

- Algorithm 1: Number of GPUs (1 vs. 2), Framework (PyTorch DDP vs. TensorFlow MirroredStrategy vs. AxoNN)
- Algorithm 2: Precision format (FP16, BF16, INT8, Distillation), all on 1 GPU
- Algorithm 3: ZeRO stage (1, 2, 3), all on 2 GPUs

Metrics collected per epoch per run:

- Epoch wall-clock time (seconds), measured with Python time.time()
- GPU memory allocated (MB), via torch.cuda.memory_allocated() for PyTorch, nvidia-smi for TF and AxoNN
- Power draw (Watts), via nvidia-smi polling every 5 seconds, logged to CSV
- Training loss and test accuracy per epoch

Energy was estimated as: Energy (Joules) = average power draw (W) x total training time (s). Power was computed by summing across all active GPUs per timestamp snapshot and averaging over intervals where GPU utilization exceeded 5%, to exclude idle periods. This methodology

follows the approach in the IEEE MICRO 2025 paper on distributed training efficiency. All experiments were run on NVIDIA A40 GPUs on the NCSA Delta supercomputer.

8. Performance Results

Algorithm 1: Data Parallelism

All experiments completed on NVIDIA A40 GPUs on NCSA Delta. Results reflect averages across 3 runs with standard deviation in parentheses.

Training Time

AxoNN was the fastest framework at both GPU counts, completing 10 epochs in 37.9s (+/-0.2s) on 1 GPU and 26.7s (+/-1.5s) on 2 GPUs. PyTorch DDP required 44.6s (+/-2.4s) on 1 GPU and 32.4s (+/-0.2s) on 2 GPUs, yielding an actual speedup of 1.38x versus the ideal 2.0x. TensorFlow required 217.2s (+/-0.1s) on 1 GPU and 138.8s (+/-4.7s) on 2 GPUs, attributable to XLA graph compilation overhead.

Test Accuracy

PyTorch DDP 1 GPU achieved the highest accuracy at 78.4% (+/-0.4%), followed by AxoNN 1 GPU at 77.9% (+/-0.7%). TensorFlow reached 75.7% (+/-2.5%) on 1 GPU and 73.7% (+/-1.7%) on 2 GPUs. AxoNN 2 GPU produced the lowest accuracy at 68.2% (+/-1.0%), a 9.7 percentage point drop consistent with stale gradient behavior from asynchronous communication at small scale.

Energy Consumption

AxoNN 1 GPU was the most energy efficient at approximately 3,537J, followed by DDP 1 GPU at 4,570J and DDP 2 GPU at 4,879J. DDP 2 GPU consumed more energy than DDP 1 GPU despite being 27% faster, because two GPUs drawing power simultaneously offset the time savings. TensorFlow consumed 44,245J on 1 GPU and 56,558J on 2 GPUs.

Algorithm 2: Mixed Precision and Quantization

All experiments on 1 GPU on NCSA Delta. FP32 baseline (DDP 1 GPU) included for comparison.

Training Time

INT8 was the fastest at 38.8s (+/-0.0s), slightly faster than the FP32 baseline of 44.6s. BF16 came in at 45.9s (+/-0.2s) and FP16 at 49.2s (+/-0.7s). Distillation was slowest at 74.1s (+/-0.3s) because each step requires a forward pass through both teacher and student.

Test Accuracy

All four methods maintained accuracy within 1% of the FP32 baseline of 78.4%. INT8 reached 78.6% (+/-0.5%), FP16 reached 77.6% (+/-1.4%), BF16 reached 77.7% (+/-0.4%), and distillation reached 77.4% (+/-0.2%). The differences are smaller than the run-to-run variance.

Energy Implications

FP16 and BF16 reduce memory bandwidth requirements by 2x. INT8 reduces the memory footprint by 4x and was the fastest training method, making it the most energy-efficient precision format tested. Distillation adds training cost but produces a smaller model that is cheaper to deploy at inference time.

Algorithm 3: ZeRO Memory Optimization

All experiments on 2 NVIDIA A40 GPUs on NCSA Delta. DDP 2 GPU baseline (32.4s, 75.5% accuracy) included for comparison.

Training Time

ZeRO Stage 1 completed 10 epochs in 45.4s (+/-1.4s) and Stage 2 in 43.8s (+/-0.4s), both slower than the DDP 2 GPU baseline of 32.4s. Stage 3 was dramatically slower at 120.7s (+/-2.4s), nearly 4x slower than DDP. The Stage 3 slowdown is caused by the all-gather operation that must reassemble model parameters before every forward and backward pass. At ResNet18 scale this communication cost dominates the actual computation time.

Test Accuracy

ZeRO Stage 2 achieved the best accuracy at 74.6% (+/-0.5%), close to the DDP 2 GPU baseline of 75.5%. Stage 3 reached 73.3% (+/-0.9%) and Stage 1 reached 72.8% (+/-1.0%). All three stages are slightly below DDP, which is expected since DeepSpeed's learning rate scheduler interacts differently with SGD than PyTorch's native scheduler.

Memory Usage

Stages 1 and 2 used 102.5MB per GPU, lower than DDP's measured memory at comparable configurations. Stage 3 used 910.5MB per GPU, substantially higher than Stages 1 and 2 due to the all-gather buffer required to hold the full parameter tensor during forward and backward passes. At ResNet18 scale, Stage 3's memory usage is counterproductive; the benefits only materialize at model sizes that exceed single-GPU memory capacity.

9. Conclusions

The results across all three algorithms confirm several expectations from the literature while surfacing findings that were not obvious going in.

For data parallelism, PyTorch DDP achieved the best accuracy at both GPU counts and maintained competitive energy efficiency. The measured speedup from 1 to 2 GPUs was 1.38x versus the ideal 2.0x, consistent with the communication overhead predicted by the complexity analysis. AxoNN at 1 GPU achieved the lowest energy consumption of any Algorithm 1 configuration at 3,537J. However, scaling AxoNN to 2 GPUs hurt accuracy significantly without a proportional energy benefit.

The most significant finding from Algorithm 1 is that adding a GPU does not reliably reduce energy consumption. DDP 2 GPU used more total energy than DDP 1 GPU despite finishing faster. Faster is not the same as greener.

For mixed precision, reducing precision from FP32 to FP16, BF16, or INT8 does not meaningfully hurt accuracy on this task while reducing memory bandwidth requirements. INT8 was the fastest and most energy-efficient precision format tested, completing training in 38.8s with 78.6% accuracy. Any of the four methods would be a reasonable choice for this workload.

For ZeRO memory optimization, the results reveal a clear scale dependency. Stages 1 and 2 add modest overhead compared to DDP (45s and 44s vs 32s) while maintaining similar accuracy. Stage 3 is counterproductive at this scale, using 4x more time and 9x more memory than Stages 1 and 2 due to parameter all-gather communication overhead. ZeRO's benefits are designed for models that exceed single-GPU memory capacity. At ResNet18 scale, the overhead outweighs any memory savings.

Taken together, the three algorithms suggest that the most energy-efficient training configuration tested is AxoNN at 1 GPU combined with INT8 post-training quantization. For distributed training specifically, PyTorch DDP with ZeRO Stage 2 offers a reasonable tradeoff between memory efficiency and training speed without the communication overhead of Stage 3.

The broader implication is that green training decisions depend heavily on scale. Techniques that reduce energy at large scale can increase it at small scale, and vice versa. This suggests that energy benchmarking should be conducted at the actual target scale rather than extrapolated from large-scale results.

References

Li, S., et al. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. VLDB, 2020. https://arxiv.org/abs/2006.15704

Singh, S., Bhatele, A. AxoNN: An Asynchronous, Message-Driven Parallel Framework for Extreme-Scale Deep Learning. IPDPS, 2022. https://arxiv.org/abs/2110.13005

Micikevicius, P., et al. Mixed Precision Training. ICLR, 2018. https://arxiv.org/abs/1710.03740

Hinton, G., Vinyals, O., Dean, J. Distilling the Knowledge in a Neural Network. NeurIPS Workshop, 2015. https://arxiv.org/abs/1503.02531

Rajbhandari, S., et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. SC'20, 2020. https://arxiv.org/abs/1910.02054

Characterizing the Efficiency of Distributed Training: A Power, Performance, and Thermal Perspective. IEEE MICRO, 2025. https://dl.acm.org/doi/10.1145/3725843.3756111

Mahmud, H. Converting Autoencoder Based Energy-Efficient and Secure DNN Inference on Edge Devices. PhD Dissertation, UTSA, 2025.

Fouda, M.E., et al. Quantized Convolutional Neural Networks: A Hardware Perspective. Frontiers in Electronics, 2025.

Sergeev, A., Del Balso, M. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. arXiv, 2018. https://arxiv.org/abs/1802.05799

The Carbon and Water Footprints of Data Centers and AI. ScienceDirect, 2025. https://www.sciencedirect.com/science/article/pii/S2666389925002788